

Volvo Cars — Data-Driven Playwright E2E Suite

Role: Senior Product Engineer (Tech Lead)

Company: Volvo Cars

Opener / Hook

"I redesigned our E2E test suite so it fetches the same live data our pages use — now tests are written once and run for any car model, market, or viewport without configuration changes."

Situation (S)

- Product detail pages serve the **same page templates** but with **varying content** depending on car model and market — the same component structure, but entirely different data.
 - Our previous Playwright suite was **hand-configured** for specific models and markets — tests had hardcoded expectations and required duplication for each new combination.
 - We covered **2 models × 2 markets × desktop only** — a tiny fraction of real traffic scenarios.
 - Scaling this meant **manually duplicating test logic**, updating large configuration objects, and maintaining assertions that would break when CMS content changed (not because anything was actually broken).
 - Tests were **flaky** and the suite was **resistant to change** — adding a new market or model required non-trivial work.
-

Task (T)

- I identified that the root cause wasn't the tests themselves — it was that they were **disconnected from the data that drives the pages**.
 - My objective: redesign the suite so it fetches live CMS data and dynamically constructs test cases, making coverage a function of configuration, not manual effort.
 - Scope: re-architecture of the full E2E suite, integration with live data APIs, mobile + desktop coverage, and integration into the deployment pipeline.
-

Action (A)

- **Initiated and led the redesign:** identified the pattern, proposed the approach, and drove implementation.
- **Integrated tests with the same CMS API endpoints** the live pages use — tests now consume real data rather than hardcoded fixtures.
- **Dynamically constructed the test suite from data:** test cases are generated at runtime based on what the data says should exist (e.g., if the data says there are 6 feature items, the test asserts 6 are rendered — no manual count to maintain).

- **Expanded coverage to mobile and desktop** — both viewports now run for every combination, with no duplication.
 - **Integrated the suite into the deployment pipeline** — tests run as part of CI/CD and gate releases.
 - Tests are now **written once and work for any model/market/viewport** — adding a new market means adding it to the data config, not rewriting test logic.
-

Result (R)

- **Coverage expanded dramatically:** from 2 models × 2 markets × desktop → any number of models and markets × mobile and desktop — the only limit is pipeline wait time.
 - **Write-once test logic:** a test only needs updating when a component changes, not when content changes.
 - **Less flaky:** content updates no longer break tests incidentally — tests break when and only when something the user would actually see is wrong.
 - **Better coverage:** mobile added, more market/model combinations covered than before.
 - **Deployment pipeline integration:** tests now gate releases, giving the team confidence when shipping.
-

Reflection (R)

- **The insight was architectural, not technical.** The problem wasn't that we needed more tests — it was that the tests were the wrong kind of thing. Coupling test expectations to live data meant they started representing truth rather than a frozen snapshot.
- **Making tests data-driven mirrors how the product works.** The pages are data-driven; the tests should be too. Aligning them reduced the gap between "what we test" and "what users actually see."
- **Test maintenance is a DX problem.** A suite that breaks on content changes trains engineers to ignore failures. Reducing that noise made the suite trustworthy again.